Introdução à Análise de Algoritmos

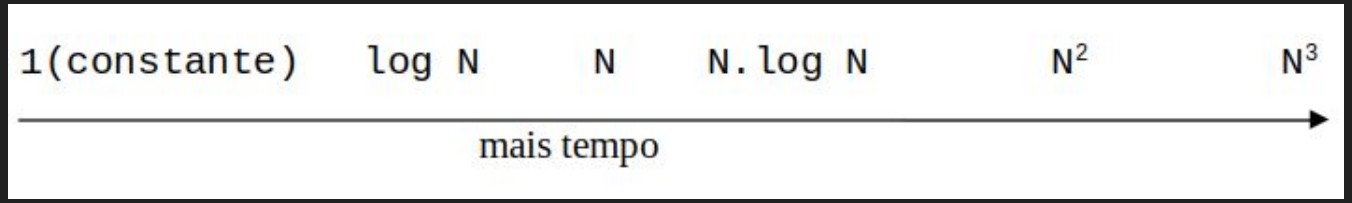
Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Introdução

- Análise de algoritmos preocupa-se com:
  - A execução de um algoritmo leva quanto tempo?
  - Como comparar dois algoritmos em questão de tempo de execução?
  - O fluxo de execução pode ser um parâmetro de análise de qualidade de um algoritmo?

- Intuitivamente: para valores de N grandes, quanto maior a ordem do expoente, mais tempo leva para o algoritmo rodar.



Relação Tempo x Execução

- O tempo de execução do algoritmo depende dos dados que ele define/manipula. Assim, o conjunto de dados está diretamente relacionado ao desempenho do algoritmo.
- Um algoritmo pode executar em um tempo reduzido se N é pequeno, mas demorar muito se N é grande ($N \sim N^3$).
- A diferença de desempenho de dois algoritmos, A1 e A2, pode não ser perceptível dependendo da máquina que o executa (especialmente se não utilizam muito CPU). Todavia, o inverso também é verdadeiro.
- Assim, **como pode-se avaliar de forma absoluta o desempenho de um algoritmo** em termos de tempo de execução?

O comportamento de alguns algoritmos

- Raízes Eq. 2o grau:

```
def raiz(a, b, c):
  ...
   x1 = (-b-sqrt(b*b-4*a*c))/(2*a)
   x2 = (-b+sqrt(b*b-4*a*c))/(2*a)
   return x1, x2
```

  - Desconsiderando casos particulares (a=0, delta negativo), sempre é realizado o mesmo número de operações -> tempo constante t = k.

O comportamento de alguns algoritmos

- Máximo entre dois elementos de uma lista:

```
def max(a, N):
    m = a[0]
    for i in range(1, N):
        if m < a[i]: m = a[i]
    return m
```

  - Conjunto de operações repetido N-1 vezes -> tempo proporcional a N + constante (t = k1 + k2 * (N-1) = **(k1 - k2) + k2 * N**
  - t proporcional a N (quando N é grande, a constante é desprezível).

Contar valor em uma lista

```
def nulos(a, N):
  c = 0
  for i in range(N):
     if a[i] == 0: c += 1
  return c
```

- Conjunto de operações repetido N vezes -> tempo proporcional a N + constante
- **t = k1 + k2 * N**
- t proporcional a N (quando n é grande, a constante é desprezível).

Contar bits significativos de inteiro

```
def num_bits(N):
  c = 0
  while N != 0:
      c += 1
      N /= 2   # ou N = N << 1
  return c
```

- c é o menor inteiro maior que lg N (base 2) -> $2^{c-1} <= N <= 2^{c}$
- t proporcional a lg N
- **t = k1 + k2 * lg(N)**

Impressão de matriz

```
def imp_tabela (N):
    for i in range(1, N + 1):
        for j in range(1, N + 1):
            print(i / j, end = "")
        print()
```

- t proporcional a N*N
- **$t = k1 + k2 * N^2$**

Multiplicação de matriz

- Como é o algoritmo de multiplicação de matriz? Quantos for’s?

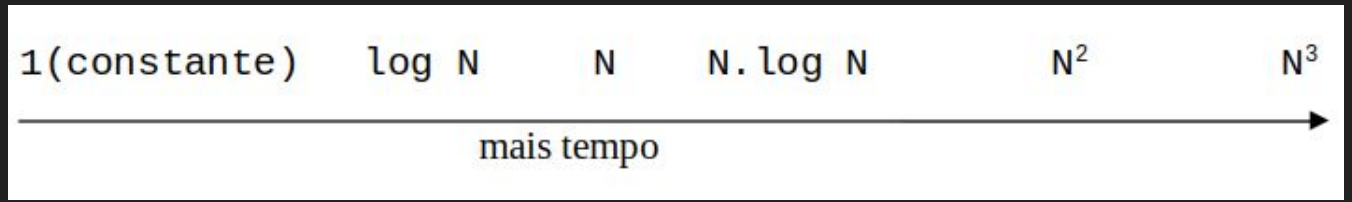
Multiplicação de matriz

- Como é o algoritmo de multiplicação de matriz? Quantos for's?
- t proporcional a N*N*N
- **$t = k1 + k2 * N^3$**

t proporcional ao número de loops aninhados

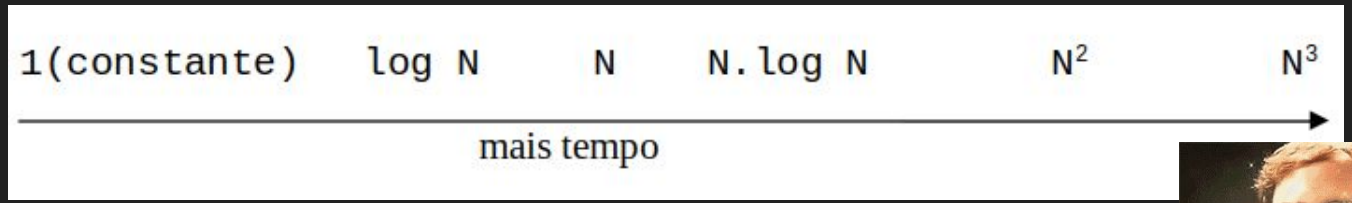
Reflexão

- t's proporcionais a:
  - 1 - sempre executa-se a mesma instrução uma vez -> t constante.
  - N - depende diretamente da entrada - número de vezes que o loop é executado -> t proporcional a N.
  - log N - base é indiferente, uma vez que log, ln e lg de N são proporcionais -> t logarítmico.
  - N^2 - 2 loops aninhados -> t quadrático.
  - N^3 - 3 loops aninhados -> t cúbico.
  - outras possibilidades: N * log N ou exponenciais (2^N, por exemplo).
- De volta a intuição:



Reflexão

- t’s proporcionais a:
  - 1 - sempre executa-se a mesma instrução uma vez -> t constante.
  - N - depende do número de vezes que o loop é executado -> t proporcional a N.
  - log N - base é indiferente, uma vez que log, ln e lg de N são proporcionais -> t logarítmico.
  - N^2 - 2 loops aninhados -> t quadrático.
  - N^3 - 3 loops aninhados -> t cúbico.
  - outras possibilidades: N * log N ou exponenciais (2^N, por exemplo).
- De volta a intuição:



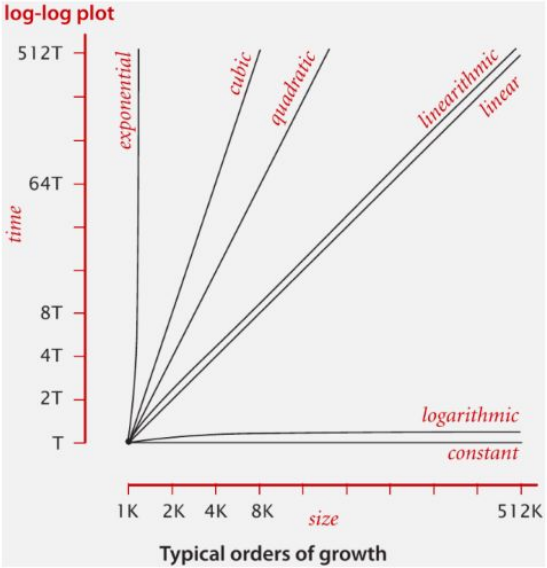


Intuição

Boas notícias: O conjunto de funções

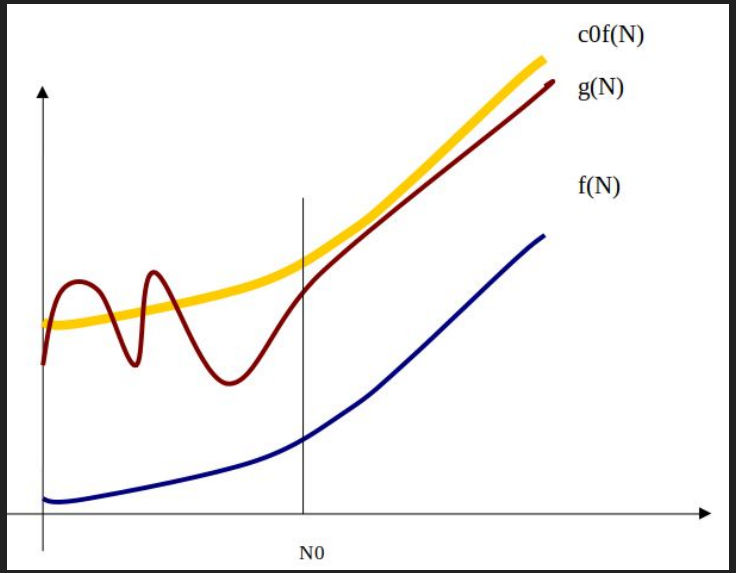
$$1,\ \log N,\ N,\ N \log N,\ N^2,\ N^3 \text{ e } 2^N$$

é suficiente para descrever a ordem de crescimento da maioria dos algoritmos mais comuns.



A notação O(f(N)) - Ordem de f(N) ou Big O

- Definição: g(N) é O(f(N)) se existem constantes c0 e N0 tais que g(N) < c0f(N) para todo N > N0.
  - O(f(N)) = { g(N) se existem constantes c0 e N0 tais que g(N) < c0f(N) para todo N>N0 }
- A partir de determinado N, a função f(N) multiplicada por uma constante é sempre maior que g(N)
- Interpretação:
  - g(N) = O(f(N)).
  - **g(N) é O(f(N)) ou g(N) ∈ O(f(N))**.



A notação O(f(N)) - Ordem de f(N) ou Big O

- c0 e N0 são importantes para entender o comportamento do algoritmo:
  - Não sabemos nada sobre $N < N0$
  - c0 pode esconder uma enorme ineficiência do algoritmo - N^2 nano-segundos é melhor que log N séculos. Ou seja, unidade N é o importante.
- **Só o termo de maior ordem é interessante. Os de ordem inferior podem ser desprezados (simplificação proposta por A. Turing)**
  - $O(1) = O(2) = O(K)$ - constante
  - $O(1 + N + N^2) = O(N^2)$
    - $1 + N + N^2 < N^2 + N^2 + N^2 = 3*N^2$ para $N > N0$
  - $O(N*\log N + N^2) = O(N^2)$
    - $N*\log N + N^2 < N^2 + N^2 = 2*N^2$ para $N > N0$

A notação O(f(N)) - Propriedades

1. O(f(N)) + O(g(N)) **é O(max{f(N), g(N)})**
2. O(f(N)) * O(g(N)) **é O(g(N)) * O(f(N))**
3. O(k*(f(N)) **é O(f(N))** desde que k diferente de 0

Exemplos de análise de algoritmos

- Algoritmo de busca sequencial:

```
# procura x em a[0], a[1], ... a[N-1]
# devolve o índice do primeiro elemento
encontrado
def busca(a, x, N):
  for i in range(N):
     if a[i] == x return i
  return -1
```

- Buscar elementos em lista exige a sua varredura.
  - Melhor caso: 1 acesso -> **x == a[0]**
  - Caso médio: (1+N)/2 acessos -> **x == aproximadamente elemento do meio da lista**
  - Pior caso: N acessos -> **x == a[n-1]**

Exemplos de análise de algoritmos

- Se considerarmos o caso médio como a média entre o melhor e o pior caso, assumimos a hipótese de a probabilidade de qualquer valor entre 1 e N estarem presentes ser a mesma. **Isso normalmente não é verdade.**
- O pior caso evidencia o comportamento e oferece um limitante superior para o tempo demandado para a execução do algoritmo.
- O caso médio é interessante, mas nem sempre é possível por depender de hipóteses adicionais sobre os dados.
- Assim, algoritmo de busca linear é O(N) (linear). Mesmo considerando a média de N/2 repetições, esse algoritmo seria O(N).

Exemplos de análise de algoritmos

- Frequência de elemento na lista:

```
def repetidos(n):
    st = str(n) # transforma n numa string
    rept = []
    # para cada dígito de n varre todos os dígitos de n
    for i in range(len(st)):
        for j in range(len(st)):
            if i != j and st[i] == st[j] and int(st[i]) not in rept:
                rept.append(int(st[i]))
    return rept
```

- Comando if repetido N * N vezes (2 loops aninhados) - logo O(N²).
- Solução alternativa O(N) - contadores de dígitos - 2 loops não aninhados.

Complexidade de Algoritmos

- O(f(N)) é a notação da complexidade de algoritmos (piores casos)
  - Esta notação ignora como o algoritmo funciona para N menores.
  - Esta notação ignora o ambiente de execução do algoritmo.
- Principais comportamentos:
  - O(1) - Constantes
  - O(N) - Lineares
  - O(N²) - Quadráticos
  - O(N^k) - Polinomiais
  - O(k^N) - Exponenciais
  - O(log N) ou O(N * log N) - Logarítmicos
- Neste curso, tentaremos responder/entender sobre complexidade ou comportamento dos algoritmos estudados

Exercícios

1. Implementar e expressar a complexidade para os casos (i) melhor, (ii) médio e (iii) pior caso para os seguintes algoritmos:
    - Raízes da equação do 2o grau
    - Máximo de uma sequência de elementos
    - Contagem de nulos em um vetor de N elementos
    - Verificação da igualdade entre dois vetores
    - Impressão matriz NxM
    - Multiplicação de matrizes
    - Produto escalar entre dois vetores



Dúvidas?

Obrigado!



Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr